

Implémentation Temps Réel d'une Commande Intelligente pour Manipulateur 2-DDL sur DSP TMS320C6713

Oula AMRAOUI, Solaimane HAKKANI, Hamza EL HAJIBI,
Scidelo Tsikomia ANDRIASAHY, et Horacia Glorieta AZONHOUMON

Filière : Génie des Systèmes Électroniques et Automatiques
ENSA : École Nationale des Sciences Appliquées de Tanger
Université Abdelmalek Essaâdi
Tanger, Maroc

Résumé—Résumé—Ce rapport présente l'étude, la simulation et l'implémentation temps réel d'une loi de commande pour un robot manipulateur à deux degrés de liberté (2-DDL). Dans un premier temps, trois stratégies d'automatique intelligente ont été développées et comparées sous environnement MATLAB/Simulink : la Logique Floue (FLC), les Réseaux de Neurones (NNC) et un PID optimisé par Algorithme Génétique (GA-PID). Les résultats numériques ayant démontré la supériorité et la robustesse du régulateur flou face aux fortes non-linéarités du système, ce dernier a été sélectionné pour la phase matérielle. L'algorithme FLC a été transcrit en langage C ANSI et déployé sur un processeur de signal numérique (DSP) TMS320C6713. Les essais expérimentaux, menés en configuration *Hardware-in-the-Loop* (HIL) et validés sur un oscilloscope physique, confirment la capacité de la carte DSP à exécuter la commande floue en respectant de strictes contraintes de temps réel, tout en assurant un suivi de consigne stable et précis.

Index Terms—Robotique, Logique Floue, Réseaux de Neurones, DSP TMS320C6713, Hardware-in-the-Loop, Temps Réel.

I. INTRODUCTION

L'évolution croissante de la robotique industrielle exige des systèmes de commande capables d'allier une grande précision de suivi de trajectoire à une robustesse infaillible face aux perturbations. Cependant, la dynamique d'un manipulateur à deux degrés de liberté (2-DDL) est intrinsèquement complexe, régie par de fortes non-linéarités issues des couplages inertiels, des forces de Coriolis et des variations de couple gravitationnel.

Face à ces défis, les approches de commande classiques, bien qu'éprouvées, montrent des limites lorsque le système s'éloigne de son point de fonctionnement linéarisé. Pour pallier ces restrictions, les techniques de commande intelligente telles que la Logique Floue (FLC), les Réseaux de Neurones Artificiels (NNC) et l'optimisation par Algorithme Génétique (GA) se sont imposées comme des alternatives de premier plan.

Si les simulations sous environnement MATLAB/Simulink permettent de valider théoriquement l'efficacité de ces approches, le véritable défi de l'ingénieur réside dans leur déploiement pratique sur des architectures matérielles embar-

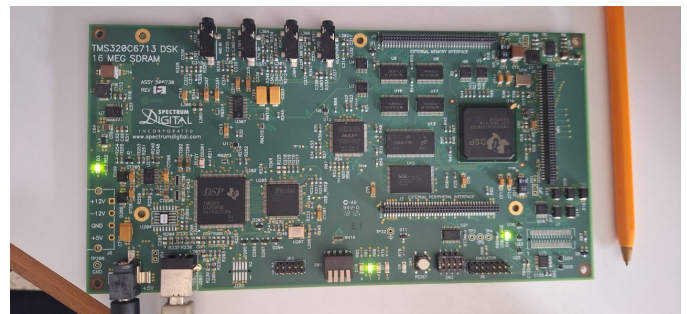


FIGURE 1. Carte DSP TMS320C6713 Laboratoire

quées, soumises à de strictes contraintes de temps d'exécution. C'est précisément l'objectif de ce présent rapport.

Dans la continuité de nos travaux de modélisation et de simulation via des S-Functions (C-MEX), ce document détaille la transition de notre architecture logicielle vers une implémentation temps réel, de type *Hardware-in-the-Loop* (HIL). La validation expérimentale a été réalisée sur un processeur de signal numérique (DSP) à virgule flottante haute performance : le **TMS320C6713** de Texas Instruments.

Afin de répondre aux exigences de cette implémentation physique, nos contributions s'articulent autour des axes suivants :

- L'adaptation et l'optimisation des algorithmes de commande (notamment le contrôleur flou) en langage C ANSI pur, en s'affranchissant des bibliothèques Simulink pour garantir une exécution autonome sur la cible matérielle.
- La configuration du pipeline matériel via Code Composer Studio (CCS) et la gestion de l'intégration numérique (méthode d'Euler) à l'intérieur du microprocesseur.
- La conversion des signaux de commande numériques via le convertisseur numérique-analogique (DAC) de la puce AIC23, permettant une visualisation physique et directe de la dynamique du robot sur un oscilloscope expérimental.

II. MODÉLISATION ET CONCEPTION DES CONTRÔLEURS

Dans cette section, nous rappelons brièvement les fondements mathématiques régissant la dynamique du robot, ainsi que les trois stratégies de commande développées pour assurer le suivi de trajectoire.

A. Modélisation Dynamique du Manipulateur à 2-DDL

La dynamique du bras manipulateur plan à deux degrés de liberté est modélisée grâce au formalisme d'Euler-Lagrange. Cette approche basée sur le bilan énergétique du système permet d'exprimer les équations du mouvement sous la forme matricielle compacte suivante :

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \quad (1)$$

où $q = [\theta_1, \theta_2]^T$ représente le vecteur des positions articulaires, $M(q)$ est la matrice d'inertie (symétrique et définie positive), $C(q, \dot{q})$ contient les termes de forces centrifuges et de Coriolis, et $G(q)$ modélise les couples de gravité. Le vecteur τ correspond aux couples appliqués par les actionneurs. Ce modèle non-linéaire et fortement couplé constitue le cœur physique que nos contrôleurs doivent maîtriser.

B. Synthèse des Stratégies de Commande

Afin de dompter ces non-linéarités, trois contrôleurs ont été synthétisés et initialement validés en simulation sous environnement C-MEX, avant leur déploiement matériel.

1) *Commande par Logique Floue (FLC)*: Un régulateur flu de type PD a été conçu pour imiter l'expertise humaine en matière de compensation d'erreurs. L'univers de discours des variables d'entrée (l'erreur e et sa dérivée $\dot{e}$) a été partitionné en cinq ensembles linguistiques (de Négatif Grand à Positif Grand). Le moteur d'inférence repose sur une matrice de 25 règles de type Mamdani, évaluées via l'opérateur *min*. La commande finale est extraite par la méthode de défuzzification du centre de gravité (barycentre), garantissant un signal de contrôle doux et robuste, idéal pour préserver les actionneurs mécaniques.

2) *Contrôleur par Réseau de Neurones (NNC)*: Afin de réduire la complexité algorithmique du FLC tout en conservant sa topographie de contrôle, un réseau de neurones à propagation avant (Feedforward) d'architecture 2-20-5-1 a été entraîné hors ligne via l'algorithme d'optimisation de Levenberg-Marquardt. Pour s'adapter à la dynamique temporelle du robot sans requérir de boucles de retard externes, ce réseau a été restructuré sous la forme d'une architecture récurrente de type NARX : la commande précédente $u(k-1)$ est réinjectée dans le vecteur d'état interne du code, offrant ainsi une véritable mémoire au contrôleur.

3) *PID Optimisé par Algorithme Génétique (GA-PID)*: En alternative aux approches purement issues de l'Intelligence Artificielle, un régulateur PID classique a été optimisé par un algorithme évolutionniste. L'Algorithme Génétique (GA) a permis d'isoler les gains optimaux (K_p, K_i, K_d) en minimisant une fonction coût globale J pénalisant à la fois l'erreur

quadratique de suivi et l'énergie de commande consommée par les moteurs :

$$J = \sum_{k=1}^N (e(k)^2 + \rho \cdot u(k)^2) \quad (2)$$

Une logique d'*anti-windup* conditionnelle a également été intégrée au code pour prévenir la saturation des intégrateurs face aux limites physiques (bridage à ± 50 Nm).

III. IMPLÉMENTATION MATÉRIELLE (ARCHITECTURE HIL)

Le passage d'un algorithme validé en simulation logicielle à son exécution sur une cible matérielle réelle exige une refonte de l'architecture du code. Cette section détaille la méthodologie adoptée pour le déploiement de notre commande sur le DSP TMS320C6713, configurant ainsi un environnement de type *Hardware-in-the-Loop* (HIL).

A. Architecture Globale

Le schéma d'implémentation repose sur une communication unidirectionnelle stricte entre l'environnement de développement et le banc d'essai physique. La figure 3 illustre cette architecture matérielle.

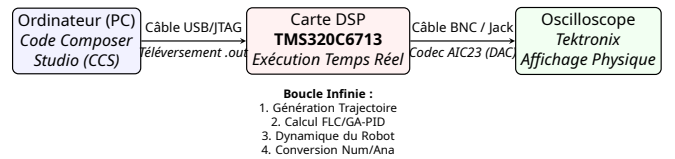


FIGURE 2. Schéma global de l'architecture d'implémentation matérielle

L'ordinateur hôte, via l'IDE Code Composer Studio (CCS), compile les algorithmes en langage machine et les téléverse dans la RAM du DSP via l'émulateur USB JTAG. Une fois programmé, le DSP s'exécute de manière totalement autonome et transmet les états dynamiques du robot à un oscilloscope physique via ses convertisseurs analogiques.

B. Adaptation Logicielle et Code C ANSI

Les S-Functions développées sous MATLAB reposaient lourdement sur la structure logicielle Simulink (pointeurs `SimStruct`, macros `mdlOutputs`, `mdlDerivatives`). Le DSP étant incapable d'interpréter ces surcouches, le code a été entièrement réécrit en C ANSI pur.

La logique d'inférence floue a été isolée dans une fonction modulaire indépendante `fuzzy_compute()`, ne prenant en argument que les états physiques (l'erreur e et sa dérivée $\dot{e}$) et les gains du contrôleur, pour retourner le couple de commande calculé. De même, les équations différentielles du robot (matrices M, C, G) ont été extraites vers une fonction `robot_dynamics()`. Afin d'exploiter pleinement l'Unité de Calcul en Virgule Flottante (FPU) du C6713 et de garantir une exécution rapide, toutes les variables de type `double` (64 bits) ont été formellement converties en `float` (32 bits).

Flux de données pour le développement et le test du TMS320C6713

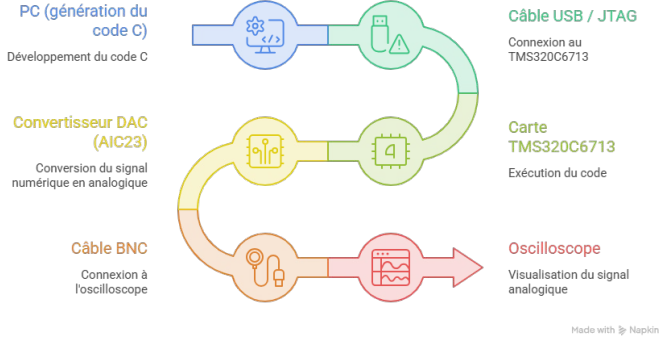


FIGURE 3. Flux de données pour le développement et le test du TMS320C6713

C. Gestion du Temps Réel et Interface DAC (AIC23)

Au cœur du DSP, un fichier `main.c` orchestre la simulation temporelle continue via une boucle d'intégration numérique d'Euler. La principale contrainte de cette implémentation réside dans la synchronisation du temps d'exécution mathématique avec le temps physique réel.

Pour ce faire, nous avons exploité la puce audio intégrée à la carte DSK (le codec AIC23). En configurant cette puce sur une fréquence d'échantillonnage de 8000 Hz, le pas d'intégration a été rigoureusement fixé à $T_s = 125 \mu s$. L'appel à la fonction d'écriture `DSK6713_AIC23_write()` agit comme un *Timer* matériel : elle bloque la boucle de calcul tant que le convertisseur numérique-analogique (DAC) n'est pas prêt à émettre le prochain échantillon, garantissant ainsi un temps réel strict.

Enfin, pour visualiser les positions angulaires θ_1 et θ_2 sur l'oscilloscope, les valeurs mathématiques (en radians) ont été amplifiées par un gain de calibrage spécifique ($K_{osillo} = 9500$) puis castées en entiers signés de 16 bits (`short`), compatibles avec le registre du DAC. Cette mise à l'échelle permet d'obtenir un affichage cohérent calibré à 50 mV/division sur l'écran matériel.

IV. RÉSULTATS EXPÉRIMENTAUX

Cette section confronte les performances théoriques obtenues lors des simulations numériques avec le comportement réel du système embarqué mesuré physiquement.

A. Rappel des Performances en Simulation

Préalablement au déploiement sur la carte DSP, les algorithmes ont été testés sous l'environnement Simulink face à des trajectoires lisses et des échelons. Le tableau I résume l'Erreur Quadratique Moyenne (MSE) mesurée pour la poursuite de trajectoire.

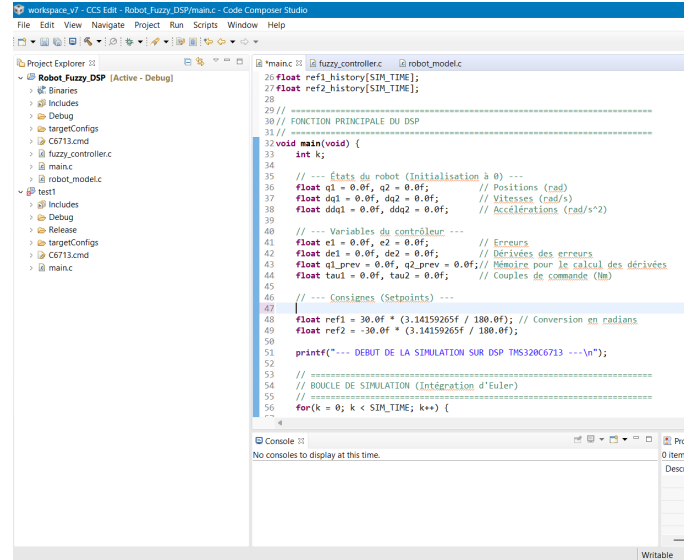


FIGURE 4. Code Composer Studio

TABLE I
COMPARAISON DES ERREURS (MSE) EN SIMULATION

Contrôleur	MSE Joint 1 (rad ²)	MSE Joint 2 (rad ²)
FLC (Mamdani)	0.0002	0.0001
NNC (NARX)	Divergence	Divergence
GA-PID	0.0774	0.0716

Les résultats numériques ont mis en évidence la supériorité de la commande par Logique Floue (FLC) face aux non-linéarités dynamiques complexes, tandis que le réseau de neurones a montré une forte vulnérabilité (divergence) en dehors de ses données d'entraînement. Le GA-PID, bien que robuste en régulation, a présenté un retard de phase sur les dynamiques rapides. Ces observations justifient la validation matérielle prioritaire du FLC.

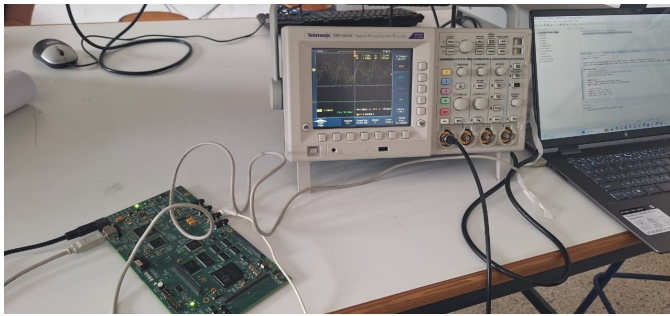
B. Validation Expérimentale sur DSP (Oscilloscope)

Pour évaluer la réactivité et la stabilité du contrôleur implémenté sur le TMS320C6713, le système a été soumis à une consigne périodique en créneaux (série d'échelons) d'une fréquence de 0.5 Hz. L'articulation 1 a été sollicitée par une amplitude de $\pm 30^\circ$, et l'articulation 2 par $\pm 15^\circ$.

Les signaux de sortie, convertis par le DAC de la carte matérielle, ont été mesurés à l'aide d'un oscilloscope Tektronix TDS 3034C. L'échelle de l'instrument a été rigoureusement calibrée avec une base de temps horizontale de 200 ms/div et une sensibilité verticale de 50 mV/div (où 3 divisions correspondent à l'amplitude maximale de 30°).

L'analyse de la Figure 5 démontre l'excellente capacité du microprocesseur DSP à exécuter la logique d'inférence floue en respectant la contrainte temps réel de $125 \mu s$. Sur le plan de l'automatique, les trajectoires physiques tracées à l'écran confirment les prédictions de la simulation :

- **Stabilité transitoire** : La réponse en échelon est nette, sans dépassement significatif (*overshoot*), validant l'ef-



D. Mise à l'Échelle et Précision Numérique

Enfin, l'interfaçage avec l'oscilloscope a imposé une contrainte de quantification. Le transfert de l'angle mathématique (flottant) vers le registre du DAC (entier `short` sur 16 bits) nécessite une mise à l'échelle. Le choix empirique du gain $K_{oscillo} = 9500$ a permis d'adapter la grandeur physique de l'angle (0.52 rad) en une tension lisible de 150 mV, garantissant ainsi l'affichage exigé de 50 mV/div (soit 3 divisions pour 30°). Cette étape prouve que l'implémentation HIL nécessite non seulement la maîtrise de l'automatique, mais également celle de l'électronique de conditionnement.

VI. CONCLUSION

Ce projet a permis de franchir toutes les étapes de conception d'un système de commande avancé, de la modélisation mathématique jusqu'à la validation matérielle. Face à la dynamique fortement non-linéaire d'un manipulateur 2-DDL, nos travaux de simulation sous MATLAB/Simulink ont d'abord permis d'évaluer trois approches distinctes : un réseau de neurones très rapide mais instable en dynamique, un GA-PID performant mais limité par sa structure linéaire, et un contrôleur par Logique Floue (FLC) offrant le meilleur compromis entre robustesse et précision.

Forts de cette analyse théorique, nous avons fait le choix exclusif de retenir la commande FLC pour l'implémentation matérielle sur la cible DSP TMS320C6713. Le passage au langage C ANSI pur et l'exploitation de l'interface audio (DAC AIC23) ont permis de valider expérimentalement cette approche. Les résultats physiques mesurés sur oscilloscope ont prouvé que la complexité calculatoire de l'inférence floue est parfaitement gérée en temps réel (fréquence de 8000 Hz) grâce à l'unité à virgule flottante (FPU) du processeur.

En guise de perspectives, cette architecture *Hardware-in-the-Loop* constitue une base solide pour de futurs développements. Il serait intéressant de l'étendre ultérieurement au GA-PID pour comparer directement l'empreinte mémoire et le temps d'exécution CPU face au FLC. À plus long terme, l'intégration d'étages de puissance permettrait de substituer l'oscilloscope par de véritables actionneurs pour piloter un manipulateur mécanique industriel.

RÉFÉRENCES

- [1] [Auteurs du papier IEEE], "[Titre de votre article de référence sur le robot 2-DDL]," [Nom du Journal ou de la Conférence], [Année].
- [2] A. Elliott, "Setting up DSP Starter Kit (DSK)," Guide d'installation et de configuration matérielle, Août 2017.
- [3] Spectrum Digital Inc., "TMS320C6713 DSK Technical Reference," Revision C. [En ligne]. Disponible : <http://c6000.spectrumdigital.com/dsk6713/revc/>
- [4] Texas Instruments, "TMS320C6000 Chip Support Library (CSL)," Réf. SPRC090. [En ligne]. Disponible : <https://www.ti.com/tool/SPRC090>
- [5] A. Elliott, "ECE436 DSK Extras (BSL/CSL Include and Lib Files)," Ressources logicielles et librairies matérielles. [En ligne]. Disponible : https://www.addisonelliott.net/ECE436_DSK_Extras.zip
- [6] Texas Instruments, "Code Composer Studio (CCS) Integrated Development Environment," Archives Documentation. [En ligne]. Disponible : http://processors.wiki.ti.com/index.php/Download_CCS